

MIT

Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

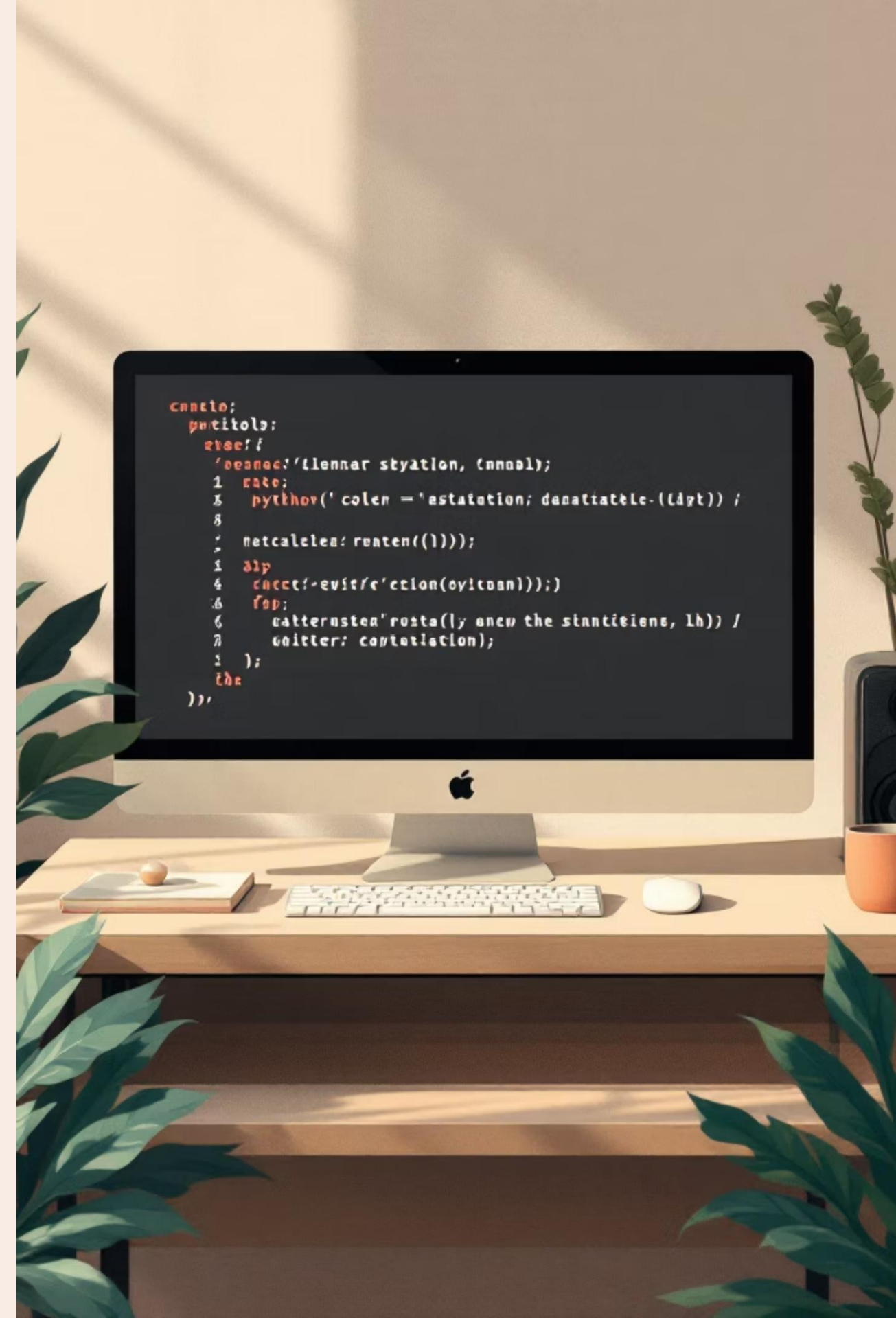
Name: Prathamesh Suresh Tandale

PRN: 202501040272

Batch: CS3-C32

EDS Theory Assignment 01

Topic: File Operations



Mastering Python File Handling

The Complete Lifecycle: Create, Read, Update, and Delete



Read



Write/Update



Delete

The Gatekeeper: The `open()` Function

```
f = open("demofile.txt", "rt")
```

Parameter 1: The File Path

Can be a local filename or an absolute path (e.g., "D:\myfiles\welcome.txt")

Parameter 2: The Mode

Dictates the action (Read, Write, Append) and format (Text, Binary)



Default Behavior Insight: By default, `open()` uses "r" (Read) and "t" (Text). Writing **`open("demofile.txt")`** is functionally identical to specifying "rt".

The Access Mode Diagnostic Matrix

Mode	Action	If File Exists...	If File is Missing...
r (Read)	Read-Only	Opens normally	Throws Error ⚠️
a (Append)	Write	Adds to end of file	Creates new file
w (Write)	Write	Overwrites entire file	Creates new file
x (Create)	Write	Throws Error ⚠️	Creates new file

Format Flags



t (Text)

Default. Opens file as a text string.

01010
01010
11101
01010

b (Binary)

Used for non-text media like images.

Architectural Safety: The Context Manager

The Old Way

```
f = open("demofile.txt")  
print(f.read())  
f.close()
```



Vulnerable. If the script crashes before reaching the close statement, the file remains locked in memory and data may be lost.

The Pythonic Way

```
with open("demofile.txt") as f:  
    print(f.read())
```



Safe. The 'with' statement automatically manages memory, locking and closing the file even if an error occurs within the block.

Reading Data: Bulk Ingestion

Macro

Inter Regular

```
print(f.read())
```

Micro-Payload

Inter Regular

```
print(f.read(5))
```

Lore ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit... Ut ne in enim in non inat, ec vommo consequat. Ut non malit evelit, surum aute dorum in magna laboris nectus ut almit, ec commodo voluptate.

Loads the entire file into memory at once. Risky for massive log files.

Lore ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit... Ut ne in enim in non inat, ec vommo consequat. Ut non malit evelit, surum aute dorum in magna laboris nectus ut almit, ec commodo voluptate.

By passing an integer to read(), you strictly limit the exact number of characters returned, preventing memory overloads.

Reading Data: Line-by-Line Iteration

The Method Call

```
print(f.readline())  
print(f.readline())
```



readline() pulls exactly one line per call.
Requires manual execution for each step.

The Pythonic Loop

```
for x in f:  
    print(x)
```

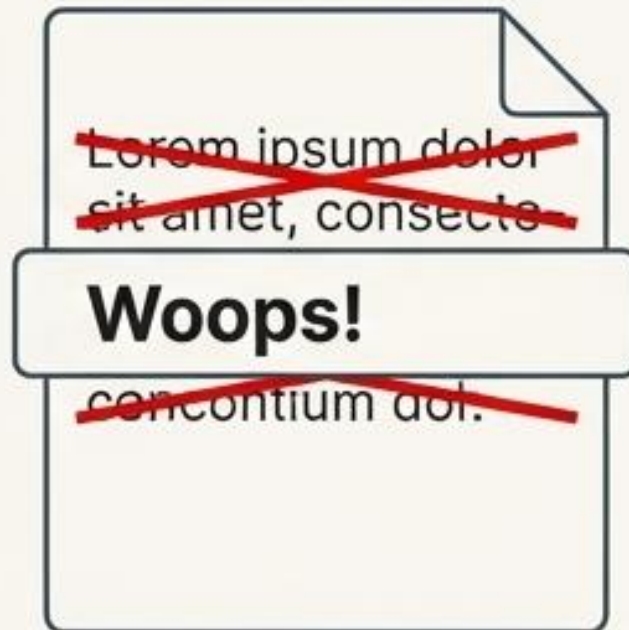


Looping through the file object directly is the most
memory-efficient and autonomous way to process
large datasets line-by-line.

Writing Data: Overwrite vs. Append

Overwrite - "w"

```
open("demofile.txt", "w")  
f.write("Woops!")
```



Destructive: The "w" parameter erases all existing content entirely before writing.

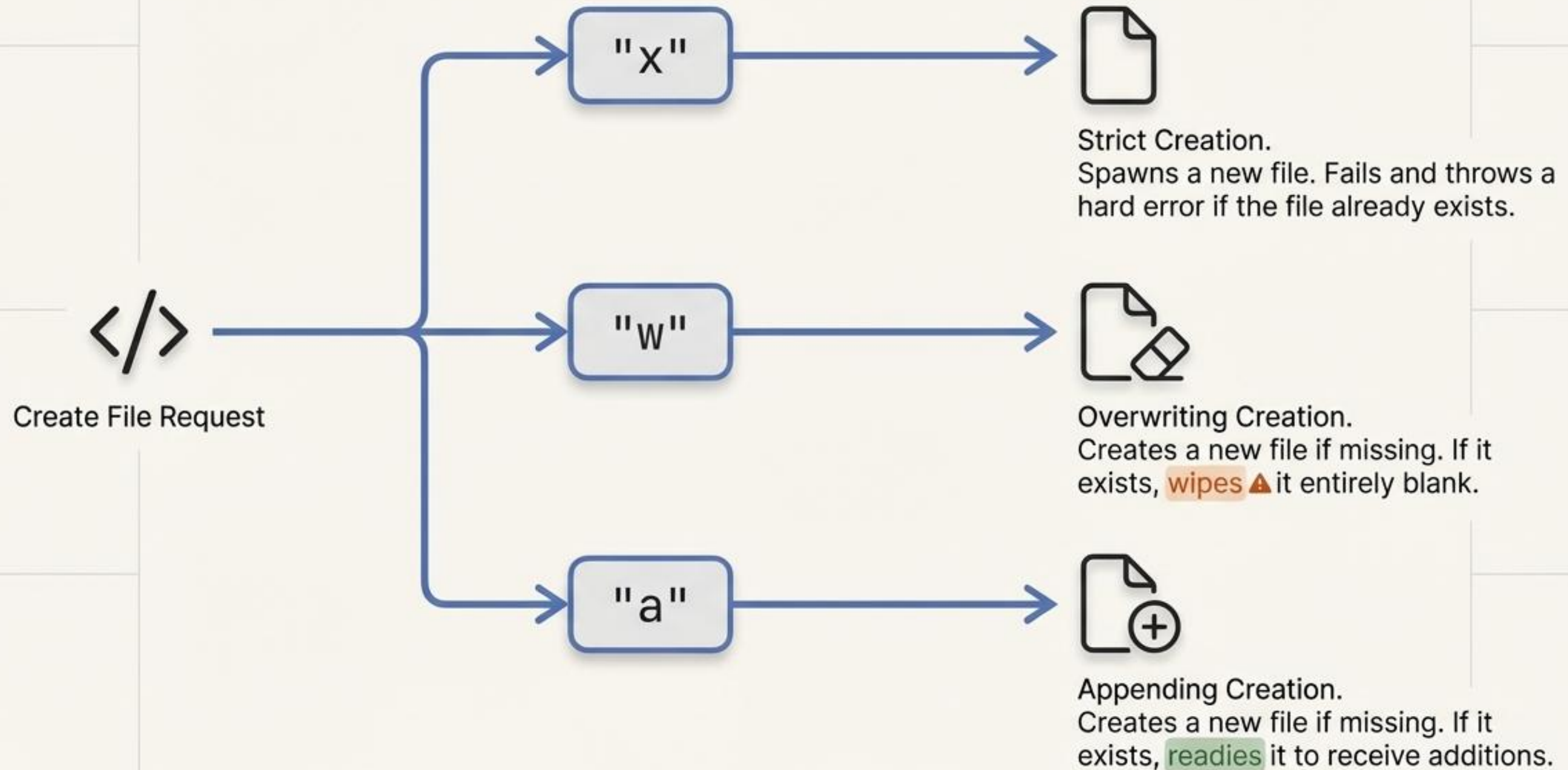
Append - "a"

```
open("demofile.txt", "a")  
f.write("More content!")
```



Additive: The "a" parameter safely tacks new data onto the very end of the file.

Creating New Files



Changing Contexts: The OS Module

Internal Scope

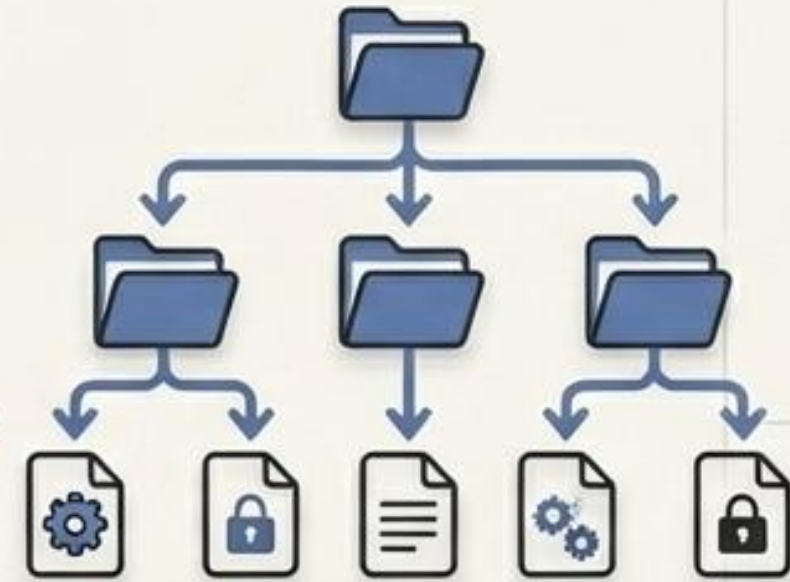


File Contents
(Managed by `open()`)

Reads and writes the data
inside the file itself.

`import os`

External Scope

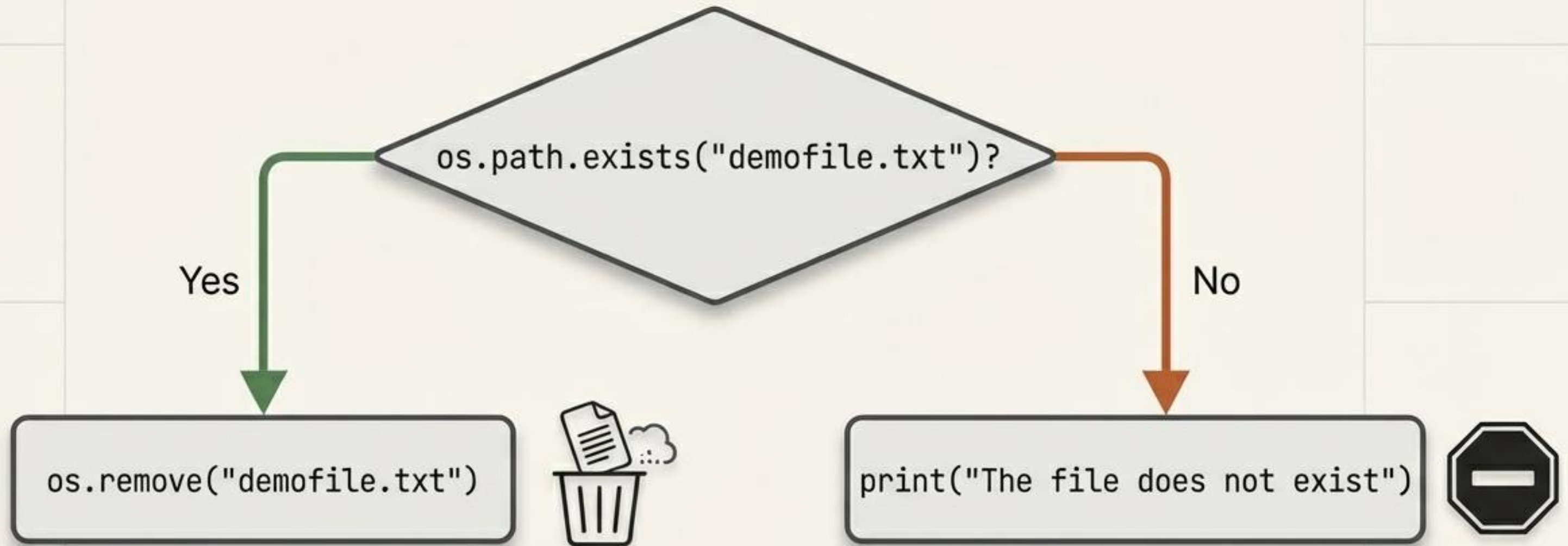


File System
(Managed by OS Module)

Handles system-level actions: renaming,
moving, and deleting files entirely.

Python requires the OS module to talk directly to your operating system to manage files at the directory level.

The Safe Deletion Flowchart



Defensive Practice: Never delete blindly. Always verify a file exists before commanding the OS to remove it to prevent fatal script crash errors.

System Cleanup: Deleting Directories

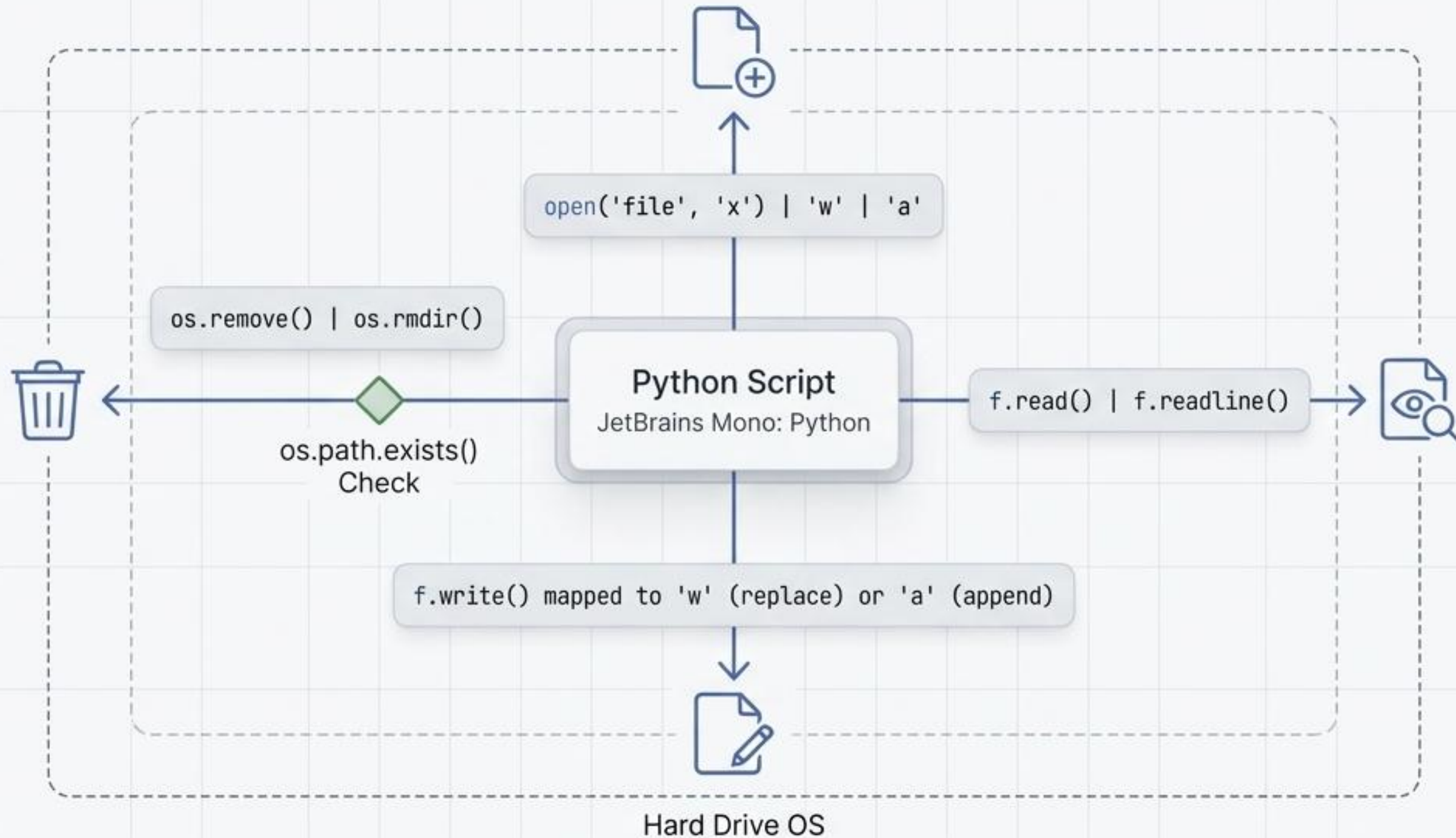
```
import os  
  
os.rmdir("myfolder")
```



The Empty Folder Rule

The `os.rmdir()` command is strict by design. It will only execute if the target directory contains zero files. This prevents the accidental mass-deletion of nested data.

The File System Command Center



The Complete Python CRUD Lifecycle. Screenshot this architectural map for your reference.